

DevOps Engineer Case Study Report

1. Practical Exercise : [Link Repo](#)

1.1. Pipeline design cần được giải thích trong report

Pipeline design của repository theo mô hình Jenkins CI kết hợp GitOps CD. Jenkins không giữ kubeconfig production để kubectl apply trực tiếp. Jenkins tạo artifact đáng tin cậy: checkout source, chạy test, render manifest, build image, scan image, push registry và cập nhật image tag trong GitOps. Sau đó ArgoCD kéo desired state từ Git để triển khai xuống Kubernetes.

- Checkout và Resolve Strategy quyết định branch/tag đang đi vào môi trường nào, image tag nào, có push image hay không và có cần approval hay không.
- Quality Gates chạy song song App Lint & Test, Helm Lint và Kustomize Render để fail sớm trước khi tốn thời gian build/push image.
- Build/Scan/Push tạo Docker image có tag immutable và label traceability, sau đó chỉ push với branch/tag được phép deploy.
- Update GitOps rewrite gitops/<env>/values.yaml, commit [skip ci] và để ArgoCD rollout thay vì Jenkins deploy trực tiếp.

1.2. Troubleshooting

Troubleshooting được chia theo layer để reviewer thấy cách debug có hệ thống, không phải chỉ liệt kê lệnh rồi rạc. Lỗi trước khi image được push thuộc CI/Jenkins; lỗi image đã push nhưng app chưa lên thuộc Kubernetes/GitOps; lỗi app đã lên nhưng chậm thuộc production incident.

- CI troubleshooting: credential GitHub, checkout, dependency install, unit test, Docker build, Trivy scan và registry push.
- Kubernetes troubleshooting: image pull, pod lifecycle, ConfigMap/Secret, readiness/liveness, Service/endpoints, Ingress/NodePort, HPA và ArgoCD sync.
- Runtime troubleshooting: logs, metrics, tracing, CPU throttling, error rate, p95/p99 latency và correlation với deployment mới nhất.

1.3. Design decisions

Các quyết định thiết kế không để thành mục tổng kết ở cuối. Mỗi quyết định được giải thích ngay tại task tương ứng: Task 0 giải thích vì sao chọn Helm/Kustomize/ArgoCD, rollback và scale; Task 1 giải thích Jenkinsfile template, branch strategy, approval và failure handling; Task 4 giải thích quyết định tối ưu; Task 5 giải thích quyết định rollback/fix khi incident.

- Chọn Helm + Kustomize thay vì copy nhiều YAML để tránh drift giữa dev/staging/prod.
- Chọn GitOps thay vì Jenkins kubectl apply trực tiếp để Git là source of truth và giảm rủi ro credential production.
- Chọn immutable image tag thay vì latest để trace commit và rollback rõ ràng.
- Chọn manual approval cho production nhưng auto-sync cho dev/staging để cân bằng tốc độ và an toàn.

2. Hướng dẫn chạy demo

Repository hiện tại đã có docker-compose.yml để chạy Jenkins Controller, Jenkins Agent, local registry và demo application. Script start-local.sh sinh SSH key cho agent, render JCasC và docker compose up stack local.

```
git clone https://github.com/MADARAKZ/devops-test.git
cd devops-test
```

```
chmod +x scripts/*.sh
./scripts/start-local.sh
# Jenkins: http://localhost:8080 admin/admin123
# Demo app local: http://localhost:8081
# Local registry: localhost:5000
```

2.1. Kết nối Jenkins với GitHub và chạy pipeline

- Tạo GitHub Personal Access Token có quyền repo để Jenkins có thể checkout/push GitOps commit.
- Trong Jenkins, thêm credential ID github-push đúng với default parameter GIT_CREDENTIALS_ID trong Jenkinsfile.
- Tạo polling job bằng scripts/create-polling-job.sh hoặc dùng Multibranch Pipeline nếu muốn tự detect develop/staging/main.

```
./scripts/create-polling-job.sh https://github.com/MADARAKZ/devops-test.git main
```

2.2. Install ArgoCD bằng Helm values

Repo có file argocd/values.yaml dùng riêng cho ArgoCD Helm chart. Setting quan trọng là configs.cm.kustomize.buildOptions: --enable-helm, vì GitOps overlay dùng Kustomize helmCharts để render chart ứng dụng.

Các cấu hình chính trong argocd/values.yaml:

- configs.cm.kustomize.buildOptions: --enable-helm để ArgoCD render Kustomize overlay có helmCharts.
- configs.params.server.insecure: true để demo local truy cập UI qua HTTP đơn giản hơn.
- server.service.type: NodePort và nodePortHttp: 30090 để mở ArgoCD UI trên Minikube.

2.3. Chạy Kubernetes local và cài ArgoCD Applications

Phần Kubernetes local dùng Minikube 3-node với Cilium CNI. Đây là setup đủ nhẹ để chạy trên máy cá nhân nhưng vẫn thể hiện được orchestration, rolling update và nhiều replica production.

```
./scripts/minikube-cilium.sh start
kubectl apply -n argocd -f argocd/project.yaml
kubectl apply -n argocd -f argocd/application-dev.yaml
kubectl apply -n argocd -f argocd/application-staging.yaml
kubectl apply -n argocd -f argocd/application-prod.yaml
```

- Dev và staging có thể auto-sync để developer nhận feedback nhanh.
- Production để manual sync/approval gate, phù hợp với môi trường cần kiểm soát release.
- Nếu không cài ArgoCD, vẫn có thể deploy trực tiếp bằng wrapper Helm trong scripts/minikube-cilium.sh deploy để chứng minh chart hoạt động.

2.3. Luồng demo end-to-end

```
git checkout -b feature/demo-change
# sửa code/test rồi push feature branch -> Jenkins test + build, chưa push image
git checkout develop && git merge feature/demo-change && git push origin develop
# develop -> Jenkins test/build/push/bump gitops/dev -> ArgoCD dev rollout
git checkout staging && git merge develop && git push origin staging
git checkout main && git merge staging && git push origin main
# main -> cần Manual Approval trước khi bump gitops/prod
```

3. Task 0 - Kubernetes Deployment (bắt buộc)

Task 0 là trọng tâm vì đề bài đánh giá khả năng deploy application lên Kubernetes, hiểu container orchestration và tích hợp CI/CD với K8s. Trong repository này, Jenkins không kubectl apply trực

tiếp vào cluster; Jenkins build image, push image và cập nhật GitOps. ArgoCD là thành phần CD kéo manifest từ Git xuống Kubernetes.

3.1. Resource Kubernetes và cấu trúc deploy

- Helm chart helm/devops-demo-app là base chart dùng chung cho Deployment, Service, ConfigMap, Ingress, HPA, PDB, NetworkPolicy và ServiceAccount.
- gitops/dev, gitops/staging và gitops/prod là overlay theo môi trường. Mỗi overlay giữ values riêng: replica, resource, NodePort, HPA/PDB và image tag.
- argocd/application-<env>.yaml định nghĩa source path và sync policy cho từng môi trường.
- Deployment chạy container từ image tag immutable; Service expose traffic; ConfigMap đưa cấu hình vào app; HPA/PDB dùng cho production readiness.

3.2. Vì sao chọn Helm + Kustomize + ArgoCD thay vì kubectl apply

kubectl apply thuận phù hợp với demo nhỏ có vài manifest. Nhưng case study yêu cầu tư duy chuẩn hóa platform cho nhiều project và nhiều môi trường. Nếu copy YAML cho dev/staging/prod, chỉ cần sửa một annotation, probe hoặc resource limit là dễ lệch cấu hình. Vì vậy tôi chọn Helm để gom phần chung, Kustomize để overlay môi trường và ArgoCD để kéo deployment từ Git.

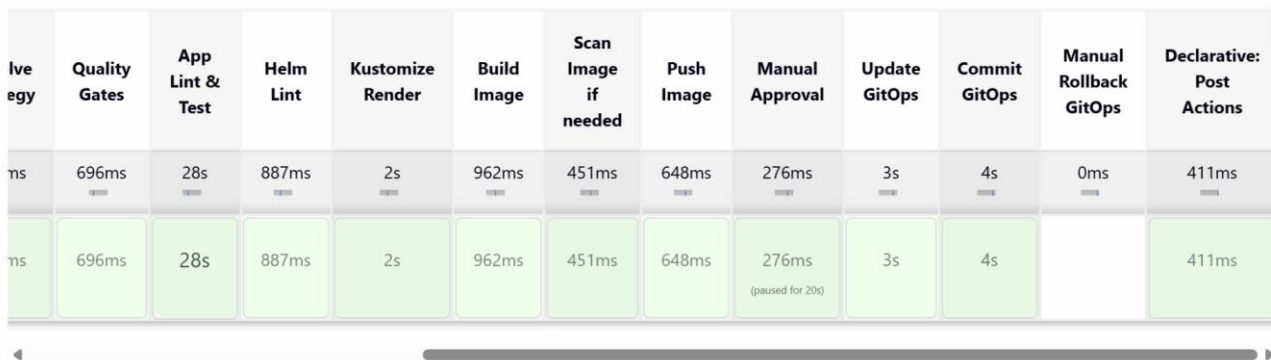
- Helm giúp template hóa resource chung, có helm lint/helm template trong CI để bắt lỗi trước khi deploy.
- Kustomize giúp mỗi môi trường chỉ ghi phần khác biệt thay vì copy toàn bộ manifest.
- ArgoCD giúp Jenkins không cần giữ kubeconfig production, giảm blast radius nếu CI bị compromise.
- Git trở thành source of truth; rollback có audit trail bằng git revert, không chỉ là thao tác thủ công trên cluster.

3.3. Luồng deploy Kubernetes

- Developer merge code vào branch tương ứng môi trường.
- Jenkins checkout, test, render manifest, build image và push registry nếu branch được phép deploy.
- Jenkins rewrite gitops/<env>/values.yaml với image.repository và image.tag mới.
- ArgoCD detect OutOfSync, render Kustomize + Helm, sau đó sync xuống namespace devops-demo-<env>.
- Kubernetes rolling update tạo pod mới; readiness pass thì Service route traffic sang pod mới.

3.4. Bằng chứng pipeline đã build/push/update GitOps

Ảnh Jenkins dưới đây thể hiện pipeline chạy qua các bước chính phục vụ Task 0: Build Image, Scan Image, Push Image, Manual Approval, Update GitOps và Commit GitOps. Đây là bằng chứng rằng deployment không chỉ nằm trên giấy mà đã được pipeline thực thi.



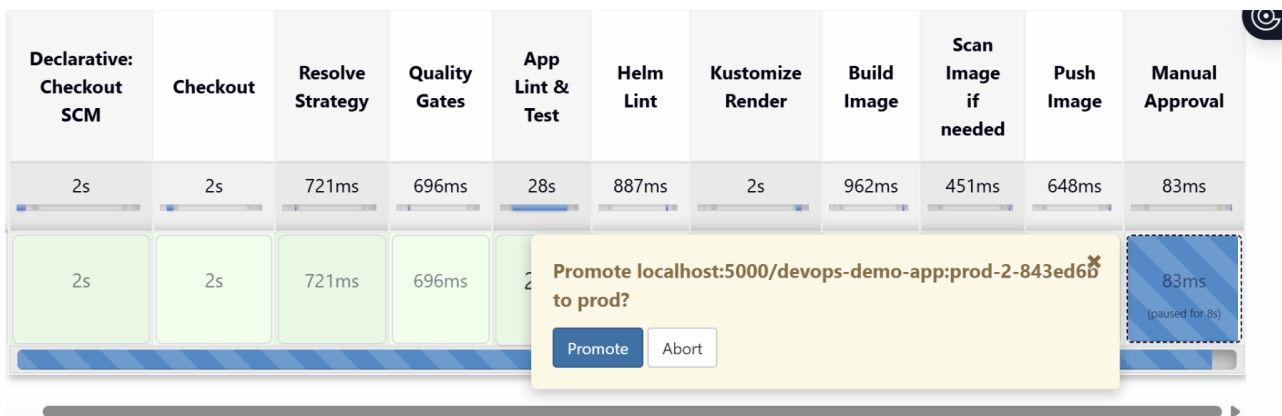
Permalinks

Hình 1. Jenkins pipeline chạy thành công qua build, scan, push và update GitOps.

- Quality Gates chạy trước Build Image để lỗi code hoặc manifest fail sớm.
- Kustomize Render giúp phát hiện lỗi Helm/Kustomize trước khi ArgoCD sync.
- Commit GitOps có [skip ci] để tránh Jenkins tự kích hoạt lại pipeline vô hạn.

3.5. Bảng chứng production có approval gate

Production không auto-promote. Với main, tag v* hoặc hotfix/*, pipeline dừng ở bước Manual Approval để người vận hành xác nhận image tag trước khi ghi vào gitops/prod.

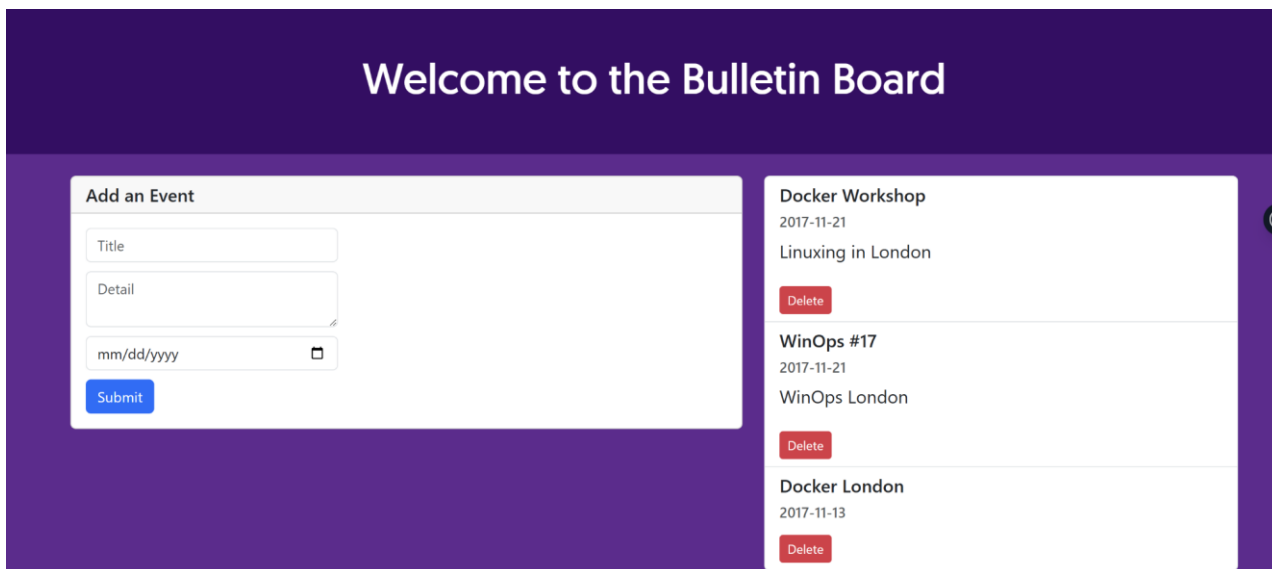


Hình 2. Manual approval trước khi promote image vào production.

- Approval gate không thay thế test; nó là lớp kiểm soát cuối sau test, scan và manifest render.
- Thông tin image tag hiển thị rõ để tránh approve nhầm version.

3.6. Bảng chứng application đã lên Kubernetes

Sau khi GitOps sync, application production được expose qua NodePort 30082. Truy cập được giao diện Bulletin Board chứng minh Service có endpoint, pod nhận traffic và application trả response thành công.



Hình 3. Demo application truy cập được qua NodePort 30082.

- Pipeline xanh chưa đủ; phải kiểm tra từ góc nhìn user-facing bằng URL thật.
- Nếu app không vào được, cần debug theo chuỗi Service selector -> endpoints -> pod readiness -> CNI/NodePort.

3.7. Bằng chứng pod production Running

Kết quả kubectl cho thấy namespace devops-demo-prod có 3 pod Running, READY 1/1 và RESTARTS bằng 0. Điều này khớp với thiết kế production có 3 replicas và rolling update đã tạo đủ pod healthy.

```
→ ~ kgpo -n devops-demo-prod
NAME                                READY   STATUS    RESTARTS   AGE
devops-demo-app-ddbbc8cd9-9zh5m    1/1     Running   0           14s
devops-demo-app-ddbbc8cd9-hvk92    1/1     Running   0           31s
devops-demo-app-ddbbc8cd9-kj9pd    1/1     Running   0           22s
→ ~ █
```

Hình 4. Production namespace có 3 pod Running, READY 1/1, restart 0.

- READY 1/1 nghĩa là container pass readiness và có thể nhận traffic.
- RESTARTS 0 nghĩa là chưa có crash/restart bất thường sau deploy.
- AGE khác nhau giữa các pod là bình thường do rolling update tạo pod theo từng thời điểm.

3.8. Config, image versioning, rolling update và scaling

- Environment/config: app nhận cấu hình qua ConfigMap, ví dụ NODE_ENV, APP_ENV và PORT. Khi ConfigMap đổi, checksum annotation trên Deployment giúp trigger rollout tự động.
- Image versioning: không dùng latest. Develop sinh dev-<build>-<sha>, staging sinh rc-<build>-<sha>, main sinh prod-<build>-<sha>, tag release dùng vX.Y.Z.
- Rolling update: production ưu tiên maxUnavailable=0 để pod cũ vẫn phục vụ traffic cho đến khi pod mới Ready.
- Scale replicas: dev có thể chạy ít replica để tiết kiệm; staging/prod tăng replica để kiểm chứng availability. Production có thể bật HPA target CPU khoảng 70%.
- PDB: production dùng PodDisruptionBudget để tránh mất quá nhiều replica khi node drain hoặc maintenance.

3.9. Rollback khi deployment lỗi

- Rollback chuẩn GitOps: git revert commit bump image trong gitops/<env>/values.yaml, push lại branch GitOps và để ArgoCD reconcile về tag trước.
- Rollback bằng Jenkins: bật parameter ROLLBACK_GITOPS=true để stage Manual Rollback GitOps tự revert commit image tag gần nhất.
- Rollback khẩn cấp bằng Helm: scripts/minikube-cilium.sh rollback hoặc helm rollback trực tiếp, sau đó vẫn phải git revert để Git và cluster không bị lệch.
- Không rollback máy móc nếu version mới đã chạy database migration không backward-compatible; khi đó phải roll-forward bằng hotfix.

3.10. Troubleshooting Kubernetes thường gặp

Khi deploy K8s lỗi, tôi không debug ngẫu nhiên mà đi theo thứ tự: image -> pod -> config/secret -> service/endpoints -> network/ingress -> ArgoCD/controller. Các lỗi hay gặp gồm:

- ImagePullBackOff / ErrImagePull: sai tag, registry không reachable, thiếu imagePullSecret, insecure registry chưa cấu hình hoặc node không resolve được registry.
- CrashLoopBackOff: app start rồi chết; xem kubectl logs --previous để kiểm tra missing env, port sai, lỗi dependency, OOMKilled hoặc command sai.
- CreateContainerConfigError: ConfigMap/Secret không tồn tại, key sai tên, volume mount sai hoặc envFrom trỏ nhầm resource.
- Pod Pending: thiếu CPU/memory, PVC chưa bind, nodeSelector không match, taint/toleration sai hoặc node NotReady.
- Readiness probe fail: pod Running nhưng không vào endpoints; kiểm tra path /health, port, initialDelaySeconds và startup time.
- Service không route: selector không match label pod, targetPort sai hoặc endpoints rỗng.
- Ingress/NodePort không vào được: sai IngressClass, path rewrite, firewall local, minikube IP hoặc Service type/port.
- HPA không scale: thiếu metrics-server, pod không khai báo resources.requests.cpu hoặc minReplicas=maxReplicas.
- ArgoCD OutOfSync/Degraded: manifest render khác cluster, resource health fail, hook fail hoặc field bị controller khác mutate.
- DNS/CoreDNS lỗi: pod không resolve được service name; kiểm tra CoreDNS pod, kube-dns service, resolv.conf trong pod và network policy chặn UDP/TCP 53.
- Permission/RBAC lỗi: ServiceAccount không đủ quyền đọc Secret/ConfigMap hoặc ArgoCD project không cho deploy namespace đích.
- Resource limit quá thấp: CPU throttling hoặc OOMKilled làm response chậm; cần so sánh requests/limits với metrics thực tế.
- Drift do sửa tay trên cluster: nếu người vận hành kubectl edit trực tiếp, ArgoCD sẽ báo drift; cách đúng là sửa GitOps repo rồi để controller reconcile.

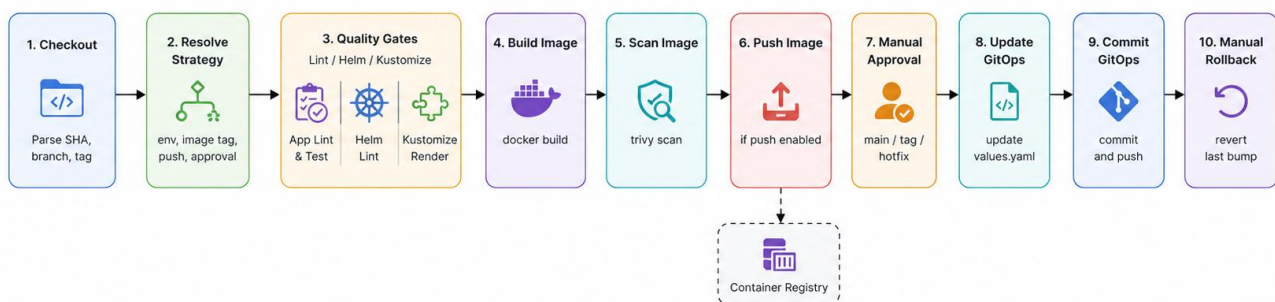
Kết luận Task 0: deployment được thiết kế theo hướng GitOps để tách rõ trách nhiệm. Jenkins tạo artifact và cập nhật desired state, ArgoCD chịu trách nhiệm apply và self-heal, Kubernetes đảm bảo rolling update/readiness/service discovery. Đây là mô hình phù hợp hơn cho môi trường nhiều team so với việc để Jenkins cầm kubeconfig production và kubectl apply trực tiếp.

4. Task 1 - Jenkins CI/CD Pipeline

Task 1 liên quan trực tiếp với Task 0. Jenkins là CI engine và cũng là điểm kích hoạt CD bằng cách cập nhật GitOps. Pipeline không deploy bằng kubectl apply trực tiếp vào production; nó tạo artifact, kiểm tra artifact và ghi desired state vào Git để ArgoCD rollout.

4.1. Sơ đồ pipeline tổng quan

Sơ đồ dưới đây tóm tắt flow chính của Jenkinsfile: checkout source, resolve strategy theo branch/tag, chạy quality gates, build/scan/push image, approval production, sau đó update GitOps để ArgoCD deploy xuống Kubernetes.



Hình 5. Sơ đồ Jenkins CI/CD pipeline: từ checkout đến GitOps update và rollback.

4.2. Pipeline structure

- Checkout: lấy source, parse SHORT_SHA, branch/tag và detect [skip ci] để tránh loop do GitOps commit.
- Resolve Strategy: quyết định target environment, image tag, có push image hay không, có cần approval hay không và GitOps branch nào được update.
- Quality Gates: chạy song song App Lint & Test, Helm Lint và Kustomize Render để fail sớm.
- Build Image: docker build từ app/Dockerfile, gắn label ci.build, ci.commit, ci.target để trace version.
- Scan Image: dùng Trivy nếu agent có cài, fail khi có HIGH/CRITICAL vulnerability.
- Push Image: chỉ chạy khi DO_PUSH=true, ví dụ develop/staging/main/tag; feature branch không push để tiết kiệm registry.
- Manual Approval: chỉ áp dụng cho main, tag v* và hotfix/* trước khi cập nhật GitOps production.
- Update/Commit GitOps: rewrite image.repository và image.tag trong values.yaml, commit [skip ci], rebase rồi push.
- Manual Rollback GitOps: revert latest bump commit khi ROLLBACK_GITOPS=true.

4.3. Branch strategy và environment mapping

Ref	Image tag	Action
feature/*	feat-<build>-<sha>	test/build only, no push, no deploy
develop	dev-<build>-<sha>	push image, bump gitops/dev, auto sync
staging/release/*	rc-<build>-<sha>	push image, bump gitops/staging
main/tag v*	prod-<build>-<sha> or vX.Y.Z	approval, bump gitops/prod

4.4. Failure handling và troubleshooting Jenkins pipeline

- `disableConcurrentBuilds()` tránh hai build cùng branch đua nhau update cùng GitOps file.
- timeout 30 phút chặn runaway build khi Docker daemon, network hoặc registry bị treo.
- set `-eu` trong shell giúp fail ngay khi command lỗi hoặc biến rỗng, tránh push nhầm image tag.
- docker login dùng `--password-stdin` và Jenkins withCredentials để giảm nguy cơ lộ secret trong log.
- git pull `--rebase` trước push để xử lý non-fast-forward khi có commit GitOps mới.
- `post.always` logout registry và prune image theo label ci.build để agent không đầy disk.
- Commit message `[skip ci]` được Jenkins đọc ngay sau checkout để tránh GitOps bump commit tạo loop CI vô hạn.

4.5. Các lỗi pipeline hay gặp và cách xử lý

- Checkout/Git credentials fail: kiểm tra credential ID github-push, PAT scope, quyền push vào branch GitOps và remote URL.
- Docker build fail vì network: dùng registry/cache nội bộ, pin base image, retry pull base image có kiểm soát.
- npm install/test fail: phân biệt lỗi dependency thật với lỗi DNS/proxy; cache npm hoặc dùng internal npm proxy khi team lớn.
- Helm lint fail: kiểm tra values thiếu key, template indent sai, quote sai kiểu dữ liệu hoặc resource name vượt giới hạn.
- Kustomize render fail: kiểm tra `--enable-helm`, chart path, valuesFile path và version kubectl/kustomize trên agent.
- Push image fail: kiểm tra registry localhost:5000 có chạy không, credential docker-registry và insecure registry nếu dùng HTTP.
- Manual approval bị treo: đây là behavior đúng với production; cần reviewer approve hoặc abort để release không tiếp tục.

Design decision của Task 1 là để pipeline xử lý policy, còn Kubernetes deployment state nằm trong Git. Jenkinsfile vì vậy có `resolveStrategy()` để gom logic branch/tag thay vì hard-code rải rác trong từng stage. Cách này giúp cùng một Jenkinsfile chạy được cho feature, develop, staging, main và tag release mà không tạo nhiều pipeline copy-paste.

- Feature branch chỉ test/build để tiết kiệm registry và tránh deploy nhánh thử nghiệm.
- Develop và staging auto-promote để feedback nhanh, vì đây là môi trường thấp hơn production.
- Main, hotfix và tag release cần approval vì tác động trực tiếp tới production.
- GitOps commit được rebase trước khi push để giảm lỗi race condition giữa các build.

5. Task 4 - Pipeline Optimization

Task 4 là scenario độc lập: pipeline hiện tại mất khoảng 25 phút, Docker build hay fail và developer phàn nàn deployment chậm. Phần này không phụ thuộc vào demo local, mà là cách tôi phân tích và tối ưu khi đưa vào môi trường team thật.

- Nguyên nhân 1 - cache kém: Dockerfile copy toàn bộ source trước khi npm ci làm invalidate dependency layer trong mọi commit.
- Nguyên nhân 2 - network/rate limit: pull image và dependency trực tiếp từ internet dễ dính timeout, proxy hoặc Docker Hub rate limit.
- Nguyên nhân 3 - pipeline tuần tự: lint, test, helm render và build chạy nối tiếp làm critical path dài.
- Nguyên nhân 4 - CI/CD dính chặt: developer phải chờ cả deploy xong mới nhận feedback, trong khi CD có thể chạy bất đồng bộ qua GitOps.

Khi gặp pipeline 25 phút, tôi không tối ưu theo cảm tính mà đo từng stage trước: checkout, dependency install, test, docker build, scan, push và deploy. Bottleneck nào lớn nhất thì xử lý trước; nếu Docker build chiếm 15 phút thì cache/layer là ưu tiên, nếu test chiếm 15 phút thì phải parallel hoặc chia test suite.

- Đo baseline bằng Jenkins stage duration, docker build --progress=plain và log thời gian npm ci/test.
- Phân biệt failure thật với failure do network; network failure cần cache/retry có kiểm soát, test failure cần sửa code.
- Không dùng retry để che test flaky; retry chỉ nên áp dụng cho bước kéo dependency hoặc push/pull registry.

5.1. Đề xuất tối ưu pipeline

- Optimized Dockerfile: copy package.json/package-lock.json trước, npm ci riêng một layer, sau đó mới copy source. Cách này giữ dependency cache khi chỉ sửa business code.
- Multi-stage image: tách build/dependencies khỏi runtime để image cuối nhỏ hơn, ít attack surface và push/pull nhanh hơn.
- .dockerignore: loại .git, node_modules local, log, coverage và file tạm khỏi build context.
- Dependency caching: dùng BuildKit cache mount hoặc cache volume cho npm/maven/gradle; trong công ty nên thêm Nexus/Harbor/Verdaccio proxy cache.
- Parallel builds: lint/test, helm lint và kustomize render chạy song song; fail-fast để developer nhận lỗi sớm.
- Artifact reuse: build image một lần, scan đúng image đó, push cùng tag đó và deploy bằng tag immutable, không rebuild lại ở stage deploy.
- Tách CI và CD: Jenkins kết thúc khi image và GitOps commit đã sẵn sàng; ArgoCD reconcile riêng, giúp feedback CI nhanh hơn và audit rõ hơn.

5.2. Kết quả kỳ vọng khi tối ưu

- Build time từ khoảng 25 phút giảm về 3-5 phút với app vừa và nhỏ nếu cache ổn định và các quality gates chạy song song.
- Docker build ít lỗi transient hơn nhờ registry/dependency proxy cache nội bộ.
- Developer nhận lỗi lint/test sớm trong 1-2 phút thay vì chờ đến cuối pipeline.
- Release an toàn hơn vì image tag immutable, scan trước deploy và GitOps commit có lịch sử rollback.

6. Task 5 - Production Incident

Task 5 là scenario độc lập: sau khi deploy version mới, response time tăng, CPU tăng và user complain. Cách xử lý đúng không phải lao vào sửa code ngay, mà là khôi phục dịch vụ trước, sau đó điều tra root cause có hệ thống.

6.1. Immediate response

- Xác định severity: ảnh hưởng bao nhiêu user, endpoint nào chậm, lỗi toàn hệ thống hay chỉ degraded.
- Mở incident channel, phân vai incident commander, communication owner và engineer điều tra.
- So sánh thời điểm bắt đầu incident với thời điểm deploy image tag mới để xác định khả năng regression.
- Tạm dừng các deployment tiếp theo vào production để tránh chồng thêm thay đổi trong lúc incident đang mở.
- Nếu có autoscaling, kiểm tra HPA đã scale chưa; nếu chưa scale do thiếu metrics hoặc request CPU sai thì rollback vẫn là lựa chọn an toàn hơn.
- Nếu impact cao, rollback ngay về image tag trước bằng GitOps revert hoặc helm rollback khẩn cấp.

6.2. Điều tra logs, metrics, tracing và xác định root cause

- Metrics: xem p95/p99 latency, error rate, RPS, CPU throttling, memory, restart count và HPA scaling behavior trước/sau deploy.
- Logs: filter ERROR/WARN theo khoảng thời gian incident; tìm timeout DB/cache, repeated log, OOMKilled hoặc request path gây lỗi.
- Tracing: lọc trace chậm để biết latency nằm ở API handler, DB query, external service call hay vòng lặp xử lý trong code.
- Kubernetes: kiểm tra kubectl describe pod, events, rollout status, endpoints và resource usage để phân biệt lỗi app với lỗi platform.
- Traceability: dùng label ci.commit/ci.build/ci.target trên image để truy về commit, PR và thay đổi code gây regression.

6.3. Rollback hoặc fix

- Rollback khi version mới gây regression nhưng schema/database vẫn backward-compatible. Cách ưu tiên là git revert GitOps bump để giữ Git là source of truth.
- Roll-forward khi migration không backward-compatible. Khi đó phải giảm traffic vào version lỗi, tạo hotfix, test staging rồi promote production.
- Sau khi service ổn định, verify lại bằng dashboard latency/error rate, endpoint health và user-facing smoke test.

6.4. Phòng tránh sự cố tương tự

- Bổ sung performance smoke test hoặc load test ngắn trước production, đặc biệt cho endpoint từng gây incident.
- Thiết lập alert: p95 latency vượt ngưỡng, error rate tăng, CPU throttling cao, restart count tăng hoặc rollout không hoàn tất.
- Dùng canary/blue-green khi hệ thống đủ trưởng thành để giảm blast radius thay vì rollout 100% ngay.
- Giữ production manual approval và release note để người approve biết image tag, commit và rủi ro thay đổi.
- Post-mortem không đổ lỗi cá nhân: ghi timeline, root cause, detection gap, prevention actions và owner cho từng action item.

6.5. DevOps metrics báo cáo leadership

- Deployment frequency: số lần deploy thành công vào production theo tuần/tháng.
- Build success rate: tỷ lệ pipeline pass trên branch chính, giúp đo sức khỏe CI.
- Lead time for changes: thời gian từ merge đến production hoặc staging deployment.

- MTTR: thời gian từ alert đến khi metric trở về baseline sau incident.
- Change failure rate: tỷ lệ deployment phải rollback/hotfix, phản ánh chất lượng release.